

Draw-to-Web

Developer Guide

Architecture, Data Flow, and Design Decisions — Version 0.2.0

A visual web-page builder that compiles a document tree into clean HTML, CSS, and JS.

Table of Contents

1. Project Overview.....	2
1.1 The problem it solves.....	2
1.2 What it does.....	2
1.3 Tech stack and why each piece was chosen	2
2. High-Level Architecture	3
3. Repository Layout	3
4. Subsystem Deep-Dives.....	4
4.1 The document model — the single source of truth.....	4
4.2 State management.....	5
The mutation path	5
Undo / redo.....	5
Active-slot routing.....	6
4.3 The Canvas editor	6
4.4 The semantic engine — inferSemantics.....	7
4.5 Code generation and export	8
4.6 Electron shell and IPC.....	10
4.7 Live Preview	10
4.8 Design tokens (cross-cutting).....	10
5. End-to-End Walkthrough	10
5.1 Insert	10
5.2 Arrange and style	11
5.3 Export	11
6. Key Design Decisions and Trade-offs	12
7. Build, Run, and Development Workflow	12
8. Known Limitations and Future Work	13
9. Glossary.....	13

How to read this guide. Every claim is grounded in the actual source; files and functions are named inline in monospace (for example `src/store/dispatch.ts`). Diagrams are rendered from the editable Mermaid sources in `docs/diagrams/*.mmd`. Open questions and incomplete wiring are called out in yellow “Flagged” boxes rather than smoothed over.

1. Project Overview

1.1 The problem it solves

Building a modern, responsive, accessible web page by hand requires fluency in HTML semantics, CSS layout, responsive breakpoints, design tokens, and accessibility rules. Draw-to-Web lets a non-coder compose a page visually on a canvas and emits clean, standards-based HTML, CSS, and (optional) JavaScript that a developer would be comfortable shipping. The visual editor is the input; a deterministic code generator is the output.

1.2 What it does

- Renders an editable page on a canvas from a single in-memory document tree.
- Lets the author insert primitives and ready-made presets, style them, and arrange them with real CSS Flexbox/Grid.
- Carries per-breakpoint and per-state (hover/focus/active) values on every element.
- Drives all colour/spacing/typography from reusable design tokens.
- Validates the document live (accessibility + structure) and blocks export on serious violations.
- Generates and bundles a complete website as a ZIP via an Electron main process.

1.3 Tech stack and why each piece was chosen

The stack is recorded in the project README and CLAUDE.md and reflected in package.json. The “why” for each major choice:

Concern	Choice	Why this and not the alternative
Desktop shell	Electron	Needs native file dialogs, local image processing (sharp), and a packaged cross-platform app. A pure web app cannot write a ZIP to disk or run sharp.
UI	React 18 + TypeScript (strict)	Component model fits a recursive canvas; strict TS catches document-shape errors at compile time. No "any" is allowed.
State	Zustand + Immer	Tiny, hook-based, and selector-driven so a leaf edit does not re-render the whole tree. Immer gives structural sharing and the patch stream used for undo/redo. Redux was rejected as heavier boilerplate for the same result.
Validation	Zod	One schema validates the document at every process boundary (file load, IPC) and is kept in lockstep with the TypeScript types.

Concern	Choice	Why this and not the alternative
Canvas drag/drop	@dnd-kit	Pointer-sensor based, accessible, and works with real DOM nodes — the canvas renders actual HTML, not a <canvas> bitmap, so a DOM-aware DnD library fits.
Layers tree	react-arborist	A virtualizing tree with built-in selection/rename, sized for large documents.
UI primitives	Radix UI	Unstyled, accessible tabs/dialog/popover/switch primitives — accessibility is a hard project requirement.
Colour	react-colorful + chroma-js	Small picker; chroma-js computes WCAG contrast for the live badge.
Output formatting	prettier, lightningcss, terser	Generated code is prettier-formatted so it reads hand-written; minifiers compress it for production.
Images	sharp	Native, fast WebP + responsive-variant generation in the main process.
Bundling/build	Vite + electron-vite + electron-builder	Fast HMR in dev; packaged installers for release.

2. High-Level Architecture

Draw-to-Web runs three contexts. The **main process** (Electron, Node) owns the application lifecycle, native file operations, the sharp image pipeline, the custom `dtw-asset://` image protocol, the minifiers, and the Live Preview window. The **renderer process** (React) holds all business logic: the UI, the Zustand stores, and the code generator. The **preload** script is a thin, typed bridge: it exposes `window.electronAPI` through `contextBridge` and is the only channel between renderer and main.

Data flows in one direction inside the renderer: **UI writes to the store; the generator reads from the store**. The canvas is a rendering of the document tree, never the source of truth. The renderer talks to main only through the preload bridge (no raw `ipcRenderer` in the renderer), and main runs with `contextIsolation: true, nodeIntegration: false, sandbox: true` (`src/main/index.ts`).

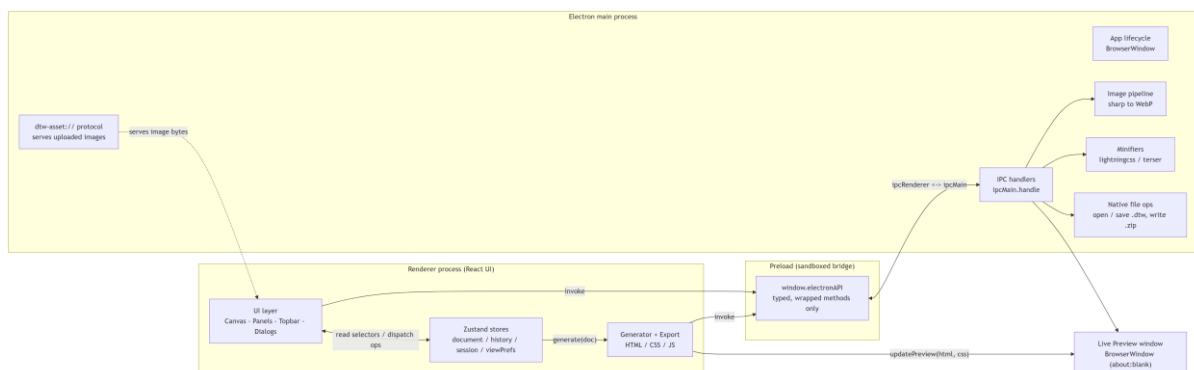


Figure 1. High-level architecture — main process, preload bridge, renderer, and the Live Preview window.

3. Repository Layout

The source tree is organised by layer. Each top-level folder under `src/` has a single responsibility and a single owner.

Path	Responsibility
src/main/	Electron main: lifecycle, BrowserWindow, IPC handlers (ipc.ts), sharp invocation (imagePipeline.ts), the dtw-asset protocol. No business logic.
src/preload/	contextBridge exposing the typed electronAPI surface. The only renderer↔main channel.
src/shared/	Types shared across processes (electronAPI.d.ts, report shapes).
src/document/	The document model: types.ts, schemas.ts (Zod), operations.ts (the only legal mutations), tokens.ts (resolveToken), validation.ts, migrations.ts, presets/.
src/store/	Zustand stores (documentStore, historyStore, sessionStore), the dispatch() entry point, styleRouting, persistence, autosave, selectors.
src/ui/	React UI: canvas/, panels/, sidebar/, topbar/, layers/, dialogs/, shortcuts/, state/ (viewPrefs).
src/generator/	Deterministic HTML/CSS/JS emission from the document tree (index.ts orchestrator, htmlEmitter, cssEmitter, jsEmitter).
src/runtime/	Self-contained vanilla-JS snippets injected into output, one per opt-in behaviour.
src/seo/	SEO + JSON-LD injection and the axe-core accessibility gate.
src/export/	The export pipeline orchestrator (index.ts), minifiers, font self-hosting, ZIP bundling.
src/templates/	Bundled starting documents (portfolio, blank, ...).
tests/	Vitest suites mirroring the module tree.

Ownership is recorded in CLAUDE.md: the Electron shell, document model, generator, runtime, SEO, and export pipeline are one owner's lane; the Zustand stores another; the entire UI a third. This matters because cross-lane changes follow a defined contract-change process.

4. Subsystem Deep-Dives

4.1 The document model — the single source of truth

Everything orbits one immutable-by-convention structure, the `Document` (`src/document/types.ts`). It carries `version`, `meta`, `tokens`, a single root `tree` of `ElementNodes`, `seo`, `runtime` flags, `variables`, `settings`, and an `assets` manifest.

There are eight primitive element types — `container`, `text`, `image`, `button`, `link`, `icon`, `list`, `divider`. Complex shapes (Hero, Cards, Nav, Footer) are **presets**: pure factories in `src/document/presets/` that compose primitives. There is no bespoke `HeroNode` type.

Every visual property is per-breakpoint via `ResponsiveProperties<T>` (`base`, optional `tablet/mobile/small`), and interactive overrides live in `states` (`hover/focus-visible/active`). The Zod schemas in `src/document/schemas.ts` mirror the types one-for-one and gate every file load.

4.2 State management

State lives in three Zustand stores plus one UI-only store. The split is deliberate: `documentStore` holds the document and a dirty flag; `historyStore` holds undo/redo patch stacks; `sessionStore` holds UI-only state (selection, active breakpoint/state, theme) and is **never serialized**; `viewPrefs` (in `src/ui/state/`) holds editor-only flags such as hover-preview and per-element hide/lock.

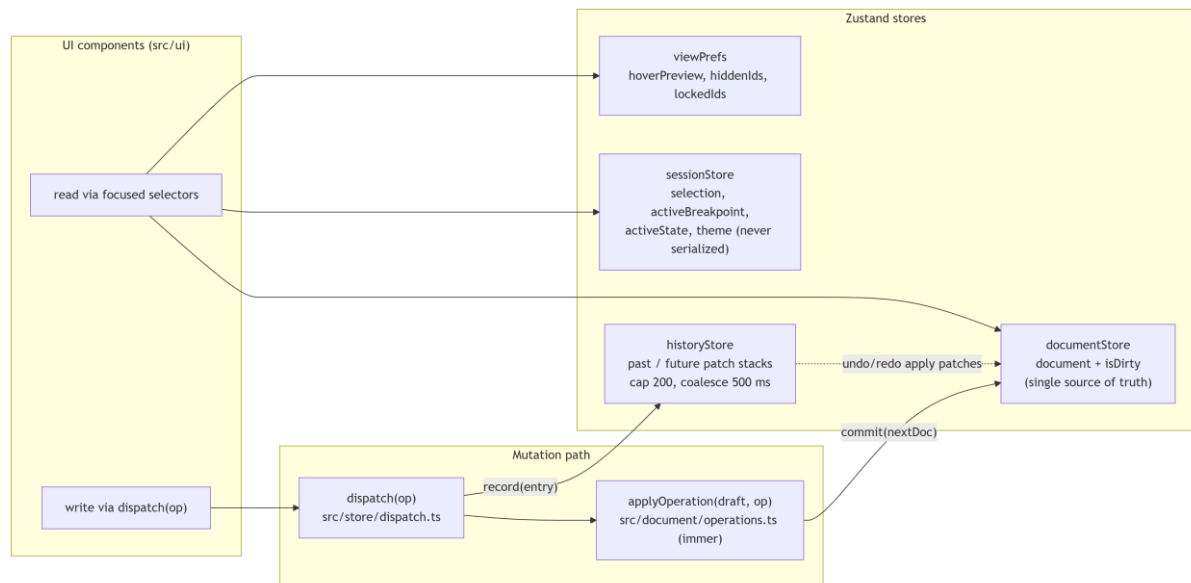


Figure 2. State management — the stores and the single mutation path through `dispatch()`.

The mutation path

Components never mutate the document directly. They call `dispatch(op)` (`src/store/dispatch.ts`), which is the only legal driver of history. The flow:

```
// src/store/dispatch.ts (shape)
export function dispatch(op: Operation): void {
  const before = useDocumentStore.getState().document
  const [after, patches, inversePatches] =
    produceWithPatches(before, (draft) => applyOperation(draft, op))
  if (patches.length === 0) return // no-op writes are dropped
  useHistoryStore.getState().record({ patches, inversePatches, label, timestamp })
  useDocumentStore.getState().commit(after) // sets isDirty = true
}
```

Operations are the only legal mutations (`src/document/operations.ts`): a discriminated union (`insertElement`, `deleteElement`, `reorder`, `updateNode`, `updateNodeStyle`, `updateNodeState`, `wrapInGroup`, token ops, `insertPreset`, ...). Each is an Immer mutator (`draft, op`) => void. Because Immer leaves untouched sub-objects referentially equal, the focused selector hooks in `documentStore.ts` (`useTree`, `useElementById`, `useTokens`) only re-render the components whose slice actually changed.

Undo / redo

History stores opaque Immer patch entries (`src/store/historyStore.ts`). `undo()` pops the last entry and the dispatcher applies its `inversePatches`; `redo()` re-applies `patches`. Two refinements: the stack is capped at 200, and sequential same-label edits within 500 ms are coalesced into one entry, so typing a word collapses to a single undo.

Active-slot routing

When the author is editing a non-default breakpoint or pseudo-state, the panel does not need to know where the value goes. `writeActiveStyle` (`src/store/styleRouting.ts`) reads `sessionStore` and routes the write: `activeState !== "default" → element.states[state]`; otherwise `→ element.style[breakpoint]`. The matching reader `resolveStyleProperty` walks the same precedence in reverse so the Properties panel displays exactly the slot it writes to.

4.3 The Canvas editor

The canvas is a recursive React renderer over the document tree, not a bitmap. `Canvas.tsx` subscribes to the tree, builds a token resolver, runs `inferSemantics`, and renders the root through `CanvasNode` (`src/ui/canvas/CanvasNode.tsx`). `CanvasNode` is wrapped in `React.memo` so an edit to one node skips untouched siblings.

Per element type it renders a real DOM node. Containers derive their tag from the inferred semantic role and host:

- `useSortable` (dnd-kit) so the node participates in its parent's sortable list (drag-reorder).
- `useDroppable` so Insert cards from the sidebar can be dropped into it.
- a `SortableContext` over its children.

Styling goes through `buildStyle.nodeStyle(node, resolve, breakpoint, state)` (`src/ui/canvas/buildStyle.ts`), which layers base `→` breakpoint `→` state and resolves token references to concrete CSS values via `resolveToken` so the canvas paints accurately. Selection lives in `sessionStore.selectedIds`; click selects, Shift/Ctrl-click accumulates, and a marquee drag on empty space selects by bounding-box intersection. Drag-and-drop for the canvas and the sidebar share one `DndContext` (`src/ui/sidebar/InsertDnd.tsx`) with a pointer-first collision strategy; dropping an Insert card resolves to the target container and index.

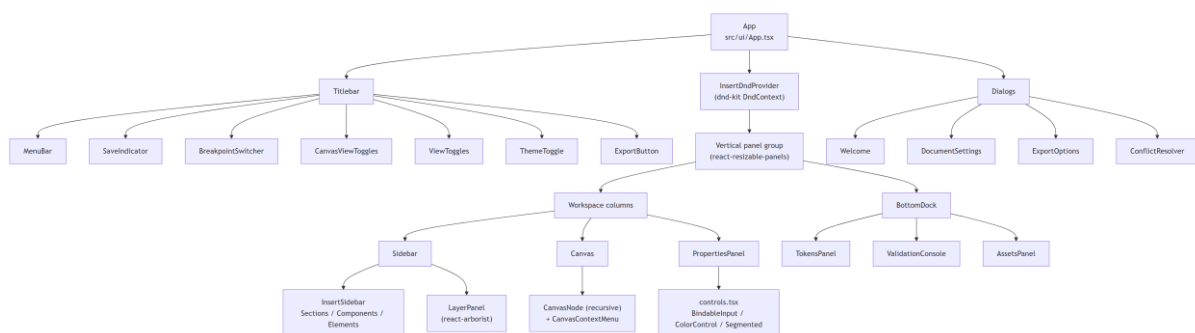


Figure 3. Renderer component hierarchy under `src/ui`.

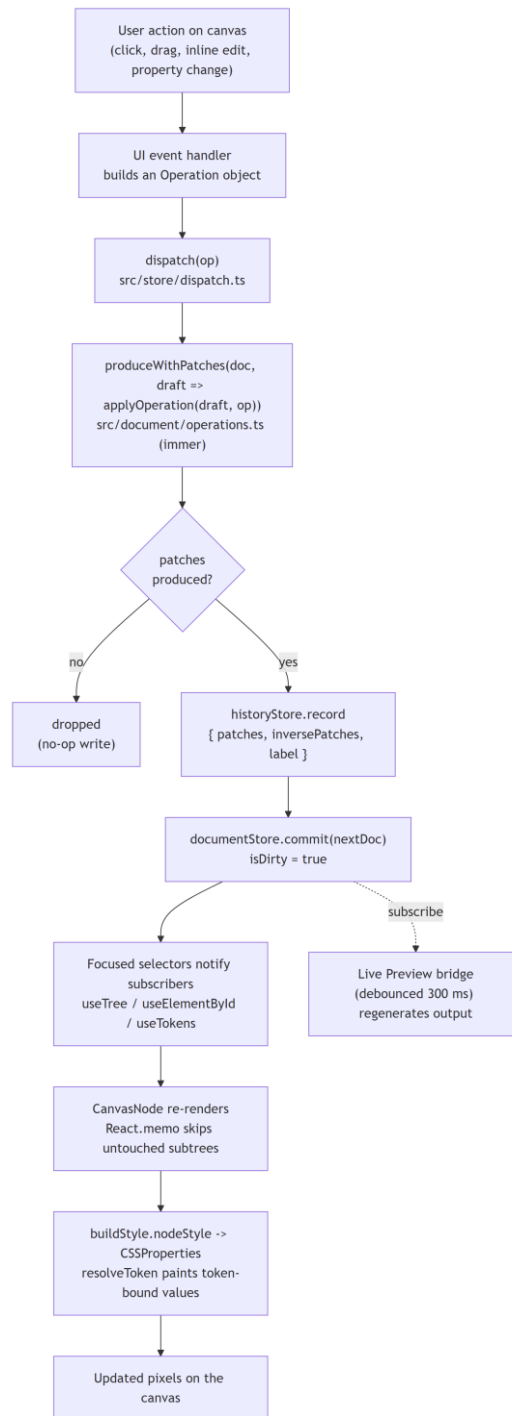


Figure 4. Unidirectional data flow: a canvas action becomes an operation, updates the store, and re-renders only the affected nodes.

4.4 The semantic engine — inferSemantics

Output quality hinges on emitting the right landmark tags (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`). `inferSemantics(tree)` (`src/ui/canvas/inferSemantics.ts`) walks the tree and fills a `semanticRole` hint on containers that do not already declare one.

Inputs: an `ElementNode` tree. Heuristics (content-gated, not purely positional):

- Depth 0 (the root container) → `main`.

- First qualifying group of two-or-more links in document order → **nav** (only the first, to avoid multiple unlabeled nav landmarks).
- A first-position depth-1 region that contains navigation → **header**.
- A last-position depth-1 region with content but no heading → **footer**.
- Other depth-1 regions → **section**; deeper generic containers stay **<div>**.

Outputs: a structurally identical tree with roles filled. It preserves explicit roles (so it is idempotent) and shares references for unchanged subtrees so memoised canvas nodes are not needlessly re-rendered.

Flagged: `inferSemantics` is currently consumed only by the live canvas (`Canvas.tsx` annotates the tree it renders). It is not applied to the document before export — the generator reads `element.semanticRole` straight off the stored tree (`htmlEmitter.containerTag = el.semanticRole ?? "div"`). In practice the exported tags come from roles that presets set explicitly; roles inferred only for preview do not reach the output. This is an inconsistency with the stated C10 “consumed by the generator” intent and is worth wiring (run `inferSemantics` in the generate/export path, or persist inferred roles).

4.5 Code generation and export

`generate(doc)` (`src/generator/index.ts`) walks the tree and returns three deterministic, prettier-formatted strings `{ html, css, js }`. It is pure: the same document always yields byte-identical output (guarded by `tests/generator/determinism.test.ts`). Three emitters do the work:

- `emitHtml` — picks each tag from the document (a container’s tag is `element.semanticRole ?? "div"`; text uses its `tag`), interpolates `{{variables}}`, and emits scoped class names `dtw-el-<id>`.
- `emitCss` — emits `:root` custom properties for every token (and a `:root[data-theme="dark"]` plus `prefers-color-scheme` block), then per-element rules scoped to `.dtw-el-<id>` that reference `var(--<prefix>-<id>)` for bound values, with `@media (max-width: ...)` blocks per breakpoint and `:hover/:focus-visible/:active` blocks per state.
- `emitJs` — concatenates only the enabled `document.runtime` snippets into one IIFE; with all flags off it emits an empty string and the HTML carries no `<script>`.

The HTML envelope is assembled by `composeHtml`: doctype, `<html lang>`, `<meta charset>`, the CSP meta, the theme FOUC guard (only when the theme-toggle runtime is on), viewport, the stylesheet link, and the deferred script link (only when JS exists).

Export pipeline. `exportProject(doc, options)` (`src/export/index.ts`) runs nine named stages and emits a progress event before each, so the UI can show which stage is running and pinpoint failures.

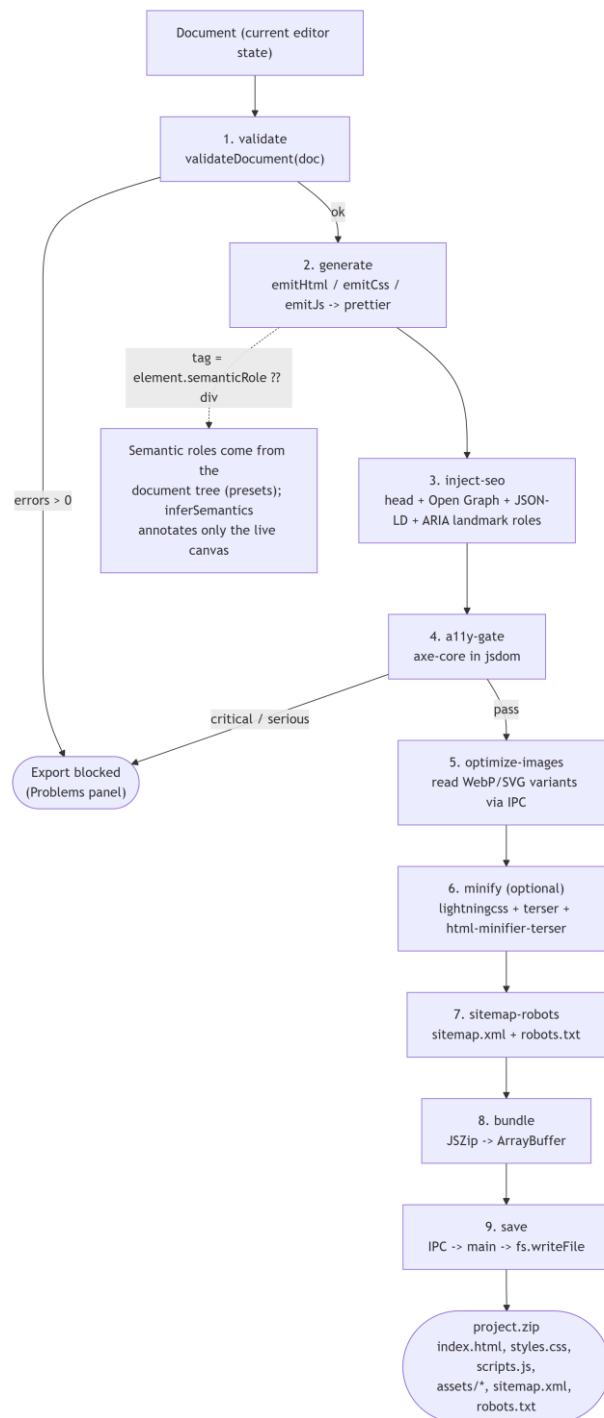


Figure 5. The nine-stage export pipeline. Validation errors and serious axe-core violations block before any bytes are written.

Notable behaviour: SEO (head tags, Open Graph, JSON-LD, ARIA landmark roles) is layered on **after** generation by `injectSEO (src/seo/index.ts)`, keeping the generator narrow. The accessibility gate runs axe-core in jsdom on the SEO-enriched HTML and blocks export on any critical/serious violation. Minification delegates to the main process from the sandboxed renderer (the minifiers are native/Node-only). A `dryRun` mode returns the strings without bundling — the basis for any code-preview surface.

4.6 Electron shell and IPC

The main process (`src/main/index.ts`) creates one secured `BrowserWindow`, injects a strict Content-Security-Policy via `onHeadersReceived`, registers the `dtw-asset://` image protocol, and wires IPC handlers (`src/main/ipc.ts`). The renderer never imports `ipcRenderer` directly; the preload script (`src/preload/index.ts`) exposes a typed surface:

```
// src/preload/index.ts (excerpt)
contextBridge.exposeInMainWorld('electronAPI', {
  saveProject, openProject, openProjectByPath,
  exportZip, uploadImage, readImageAssets,
  minifyHtml, minifyCss, minifyJs,
  openPreviewWindow, updatePreview, onPreviewClosed,
  onFileChanged, listRecentFiles, watchProject, ...
})
```

Handlers validate their inputs (buffer size, path sanitisation), write `.dtw` / `.zip` files, run `sharp`, and serve image variants. Persistence is coordinated on the renderer side (`src/store/persistence.ts`): `save` stringifies the document and clears the dirty flag on success; `open` parses JSON, runs `migrate()` (which ends in `documentSchema.parse`, so it doubles as validation), hydrates the document store, and clears history + selection.

4.7 Live Preview

Live Preview is a second `BrowserWindow` owned by main. The renderer bridge (`src/ui/preview/livePreview.ts`) subscribes to `documentStore`, debounces 300 ms, runs `generate(doc)`, and ships `{ html, css }` into the window through the `updatePreview` IPC. Opening attaches the subscription; the `onPreviewClosed` IPC tears it down. Generator hiccups are caught and logged so a transient error leaves the previous frame on screen instead of crashing the window.

4.8 Design tokens (cross-cutting)

Tokens are first-class on the document (`document.tokens`, categorised into color/spacing/fontSize/fontFamily/lineHeight/radius/shadow). Element properties are `Bindable<T>` — either a raw value or a `TokenRef` string like `color.accent.resolveToken(tokens, ref, theme)` (`src/document/tokens.ts`) is the single resolver: colour tokens pick the theme variant, others are theme-invariant. The canvas calls it to paint; the generator does **not** — it emits `var(--color-accent)` so themes can flip at runtime. Renaming a token rewrites every binding in one operation so undo treats it as a single step.

5. End-to-End Walkthrough

To make the whole system click, trace one concrete example: the author drops a Hero section and a Button, tweaks them, and exports.

5.1 Insert

Dragging the Hero card from the sidebar ends a dnd-kit drag over the canvas root.

`InsertDnd.handleDragEnd` builds an `insertPreset` operation and calls `dispatch`.

`applyInsertPreset` looks the factory up in `presetsRegistry`, materialises a subtree of primitives (a

`container` with a heading `text`, a paragraph `text`, and a `button`), and splices it under the root — all inside one `produceWithPatches` call, so undo removes the whole subtree in one step.

5.2 Arrange and style

The author selects the heading (click → `sessionStore.selectedIds`), the Properties panel renders, and they pick a font-size token. The control calls `writeActiveStyle(id, ["typography", "fontSize"], "fontSize.h1");` at the base breakpoint this dispatches `updateNodeStyle` into `element.style.base`. The canvas re-renders just that node and `resolveToken` paints the resolved pixel size. Switching to the Mobile breakpoint and changing the size again routes to `element.style.mobile`, leaving the desktop value intact.

5.3 Export

Clicking Export opens the options dialog, then runs `exportProject`. Validation passes, `generate(doc)` emits the three files, `injectSEO` adds head tags + landmark roles, axe-core passes, and JSZip bundles `index.html`, `styles.css`, and (only if a runtime flag is on) `scripts.js`. The representative output shape:

```
<!-- index.html (envelope from composeHtml) -->
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; ..." />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="stylesheet" href="styles.css" />
    <title>My Site</title>
    <!-- description, Open Graph, JSON-LD injected here -->
  </head>
  <body>
    <main class="dtw-el-root">
      <section class="dtw-el-hero" role="banner">
        <h1 class="dtw-el-title">Welcome</h1>
        <p class="dtw-el-sub">Draw it. Ship it.</p>
        <button class="dtw-el-cta" type="button">Get started</button>
      </section>
    </main>
  </body>
</html>
```

```
/* styles.css (token block + scoped, token-referencing rules) */
:root {
  --color-bg: #ffffff;
  --color-accent: #0066ff;
  --font-size-h1: clamp(28px, 2rem + 2vw, 56px);
}
:root[data-theme="dark"] { --color-bg: #0b0b10; --color-accent: #5599ff; }

.dtw-el-title { font-size: var(--font-size-h1); }
.dtw-el-cta { background: var(--color-accent); }

@media (max-width: 768px) {
  .dtw-el-title { font-size: 32px; } /* the mobile override */
}
```

The snippet shapes are exact for the envelope (`composeHtml`) and the CSS structure (`cssEmitter: :root` tokens, `.dtw-el-<id>` rules, `var(--<prefix>-<id>)`, `@media` overrides); element ids and content are illustrative.

6. Key Design Decisions and Trade-offs

- **Document tree is the only source of truth.** The canvas renders it; the properties panel mutates it through operations; the generator walks it. Rejected: letting the canvas own positions/state, which would desynchronise preview from output.
- **Composition over enumeration.** Hero/Nav/Footer are presets composed from eight primitives, not new node types. Adding a preset is a registry entry with no UI or generator changes.
- **Real DOM canvas, not a `<canvas>` bitmap.** Elements are actual HTML styled with Flexbox/Grid (never `position: absolute`), so what you see is what the generator emits, and a DOM-aware DnD library applies. Trade-off: layout math is the browser's, not ours.
- **Immer patches for undo.** Captured once in the dispatcher so history stays atomic with the document update; coalescing keeps typing to one undo step.
- **Tokens emit as CSS variables, not resolved values.** Output keeps `var(--name)` so runtime theme switching works; the canvas resolves locally only to paint.
- **Accessibility as a hard gate.** axe-core blocks export on critical/serious violations; one `<h1>`, alt text, and visible focus are enforced rather than advised.
- **Minify in main, not renderer.** lightningcss/terser are native/Node-only; the sandboxed renderer delegates via IPC. The packaging classification of these deps is a known footgun (they must be runtime dependencies, not dev).
- **Determinism.** No timestamps or random ids in generated output, enforced by a byte-equality test, so diffs are meaningful and caching is safe.

7. Build, Run, and Development Workflow

Common scripts (package.json):

Command	What it does
<code>npm run dev</code>	Start Electron + Vite in dev with hot reload (<code>electron-vite dev</code>).
<code>npm run compile</code>	Build the renderer + main bundles (<code>electron-vite build</code>) without packaging.
<code>npm run build</code>	Full production build + electron-builder installer.
<code>npm run test</code>	Run the Vitest suite.
<code>npm run typecheck</code>	tsc over the main/preload project.
<code>npm run typecheck:web</code>	tsc over the renderer project (separate tsconfig).
<code>npm run lint</code>	ESLint + prettier --check.

Two tsconfigs. Main/preload and the renderer compile under separate TypeScript projects; renderer changes must pass `npm run typecheck:web` as well as the standard typecheck.

Pre-push gate. A husky hook runs lint + typecheck + test on every push and aborts on failure; the same three run in CI. Feature branches are taken off `main` and squash-merged.

8. Known Limitations and Future Work

- **Inferred semantic roles do not reach export.** See the flag in §4.4 — `inferSemantics` annotates only the live canvas; the generator reads stored roles. Wiring it into the `generate/export` path would let inferred landmarks appear in output.
- **Layers tree drag-reorder.** The `react-arborist` tree is configured without drag; reordering is done on the canvas or via `z-order` commands. Tree drag-reorder is a natural extension.
- **Landing and Resume templates are placeholders.** The Welcome dialog lists them as “Coming soon”.
- **Document-level settings are not individually undoable.** They commit through `commitDocumentPatch` (Zod-validated) rather than the `operation/history` path; a dedicated `updateDocument` operation would add undo coverage.
- **Minifier dependency classification.** `lightningcss/terser/html-minifier-terser` must be runtime dependencies or `export-with-minify` fails at runtime and in packaged builds.

9. Glossary

Document — The single in-memory data structure the whole app reads/writes; the source of truth.

ElementNode — A node in the document tree — one of eight primitive types.

Preset — A pure factory composing primitives into a ready-made block (Hero, Nav, ...), registered in `presetsRegistry`.

Operation — A discriminated mutation applied to the document via an Immer draft; the only legal way to change the document.

dispatch(op) — The single entry point that applies an operation, captures patches, and records history.

Token — A named, reusable design value (colour/spacing/etc.) referenced from elements via a `TokenRef` and emitted as a CSS variable.

Bindable<T> — A property value that is either a raw value or a `TokenRef`.

Breakpoint / State slot — Per-screen-size (base/tablet/mobile/small) and per-pseudo-state (hover/focus/active) override locations on an element.

inferSemantics — The adapter that assigns landmark roles (header/nav/main/section/footer) to containers.

Runtime snippet — A self-contained vanilla-JS behaviour injected into output only when its `document.runtime` flag is on.

Dry run — An export that returns the generated strings without bundling/saving.

.dtw — The saved project file (JSON document), validated by Zod on load.